

# How Dual Numbers Can Be Used To Compute Derivatives

## What Are Dual Numbers?

Dual numbers are numbers of the form

$$a + b\epsilon,$$

where  $(a, b \in \mathbb{R})$ , and  $\epsilon$  is a special quantity satisfying

$$\epsilon^2 = 0, \quad \epsilon \neq 0.$$

They are analogous to complex numbers, except that while complex numbers use  $(i^2 = -1)$ , dual numbers use  $(\epsilon^2 = 0)$ . This simple algebraic rule makes dual numbers useful for automatically computing derivatives.

For the purposes of this program, the most important fact is that a dual number stores two pieces of information at once:

- a real value  $a$
- a derivative-related coefficient  $b$ .

## Computing Derivatives Using Dual Numbers

Traditionally, the derivative of a function is defined by the limit

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

From this definition, there are two common computational approaches: symbolic differentiation and numerical differentiation.

### Symbolic Differentiation

Symbolic differentiation applies derivative rules directly and keeps the result in algebraic form. For example,

$$\frac{d}{dx}(x^2) = 2x.$$

This is useful when we want an exact expression for the derivative, but symbolic computations can become increasingly complicated for large expressions. This growth in complexity is often called **expression swell**.

## Numerical Differentiation

Numerical differentiation approximates a derivative by choosing a small real number ( $h$ ). A common approximation is the forward-difference formula:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

Another common choice is the central-difference formula:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

These methods are easy to use, but they are only approximations. If  $h$  is too large, the approximation is inaccurate. If  $h$  is too small, floating-point round off error becomes significant.

This is where **machine epsilon** becomes relevant. A standard practical definition is:

**Machine epsilon is the difference between 1 and the next largest representable floating-point number.**

In other words, it measures the spacing between floating-point numbers near 1. For floating-point types,

- for floats, machine epsilon is approximately  $2^{-23}$ ,
- for double, machine epsilon is approximately  $2^{-52}$ .

Since doubles has more precision than floats, its machine epsilon is smaller.

## Dual Numbers and the Taylor Expansion

Dual numbers provide a third approach: they compute derivatives exactly at the level of arithmetic, without symbolic manipulation and without finite-difference approximation. This is a form of **automatic differentiation**.

The key idea comes from the Taylor expansion:

$$f(x + h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f^{(3)}(x) + \dots$$

Now replace  $h$  with  $b\epsilon$ . Then

$$f(a + b\epsilon) = f(a) + b\epsilon f'(a) + \frac{(b\epsilon)^2}{2!}f''(a) + \dots$$

Because  $\epsilon^2 = 0$ , every term involving  $\epsilon^2$  or higher powers vanishes. So this simplifies to:

$$f(a + b\epsilon) = f(a) + bf'(a)\epsilon.$$

This has the same form as a dual number:

$$A + B\epsilon,$$

where

$$A = f(a), \quad B = bf'(a).$$

So when we evaluate a function on a dual number, the real part gives the function value, and the coefficient of  $\epsilon$  gives derivative's value.

If we choose ( $b = 1$ ), then:

$$f(a + \epsilon) = f(a) + f'(a)\epsilon.$$

This is the form used in the program. The coefficient of  $\epsilon$  is exactly the derivative  $f'(a)$ .

## Partial Derivatives

This idea also extends naturally to functions of several variables. For a function  $f(x, y)$ ,

$$f(x + \epsilon, y) = f(x, y) + \frac{\partial f}{\partial x}(x, y)\epsilon,$$

and

$$f(x, y + \epsilon) = f(x, y) + \frac{\partial f}{\partial y}(x, y)\epsilon.$$

So by perturbing one variable at a time with  $\epsilon$ , the program can compute partial derivatives directly from the infinitesimal part of the result.